Gaussian Naive bayes

Gaussian Naive bayes is named after the Gaussian distributions that represent the data in the **training dataset**
Initial guesses are called **Prior Probabilities**

Note: When we get really small numbers, it is a good idea to take the log() of everything to prevent something called **underflow**.
Underflow = Every very computer has a limit to how close a number can get to 0 before it can no longer accurately keep track of that number.

Note: any log will do, but the natural log (ln) or log base e, is the most commonly used log() in statistic and machine learning.

$$P(x_i|y) = \frac{1}{\sigma\sqrt{2\pi}}\exp(-\frac{(x-\mu)^2}{2\sigma^2})$$

Where:

- y: class
- x_i is the feature value,
- μ is the mean of the feature values for a given class ykyk,
- σ is the standard deviation of the feature values for that class,
- π is a constant (approximately 3.14159),
- exp is the base of the natural logarithm.

It **assumes** that each feature follows a **Gaussian Distribution**.
For example in tasks like **spam detection, medical diagnosis** or **predicting house prices** where features such as **age**, **income** or **height** fit a normal distribution there Gaussian Naive Bayes can make accurate predictions.

Practical Example

To understand how Gaussian Naive Bayes works here's a simple binary classification problem using one feature: petal length.

Petal Length (cm)	Class Label
1.4	0 (Iris-setosa)
1.3	0 (Iris-setosa)
1.5	0 (Iris-setosa)
4.5	1 (Iris-versicolor)
4.7	1 (Iris-versicolor)
4.6	1 (Iris-versicolor)

We want to classify a new sample with **petal length = 1.6 cm**.

1. Separate by Class

- $Class0 : [1.4, 1.3, 1.5]$
- $Class1 : [4.5, 4.7, 4.6]$

2. Calculate Mean and Variance

For class 0:

- $\mu_0 = \frac{1.4+1.3+1.5}{3} = 1.4$
- $\sigma_0^2 = \frac{(1.4-1.4)^2+(1.3-1.4)^2+(1.5-1.4)^2}{3} = 0.0067$

For class 1:

- $\mu_1 = \frac{4.5+4.7+4.6}{3} = 4.6$
- $\sigma_1^2 = \frac{(4.5-4.6)^2+(4.7-4.6)^2+(4.6-4.6)^2}{3} = 0.0067$

3. Gaussian Likelihood

The Gaussian PDF is:

$$P(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

For $x = 1.6$:

Class 0

- $P(1.6|C=0) \approx \frac{1}{\sqrt{2\pi \cdot 0.0067}} \cdot e^{-\frac{(1.6-1.4)^2}{2 \cdot 0.0067}} \approx 0.247$

Class 1

- $P(1.6|C=1) \approx \frac{1}{\sqrt{2\pi \cdot 0.0067}} \cdot e^{-\frac{(1.6-4.6)^2}{2 \cdot 0.0067}} \approx 0$

4. Multiply by Class Priors

Assume equal priors:

$$P(C=0) = P(C=1) = 0.5$$

Then:

- $P(C=0|x) \propto 0.247 \cdot 0.5 = 0.1235$
- $P(C=1|x) \propto 0 \cdot 0.5 = 0$

5. Prediction

Since $P(C=0|x) > P(C=1|x)$,

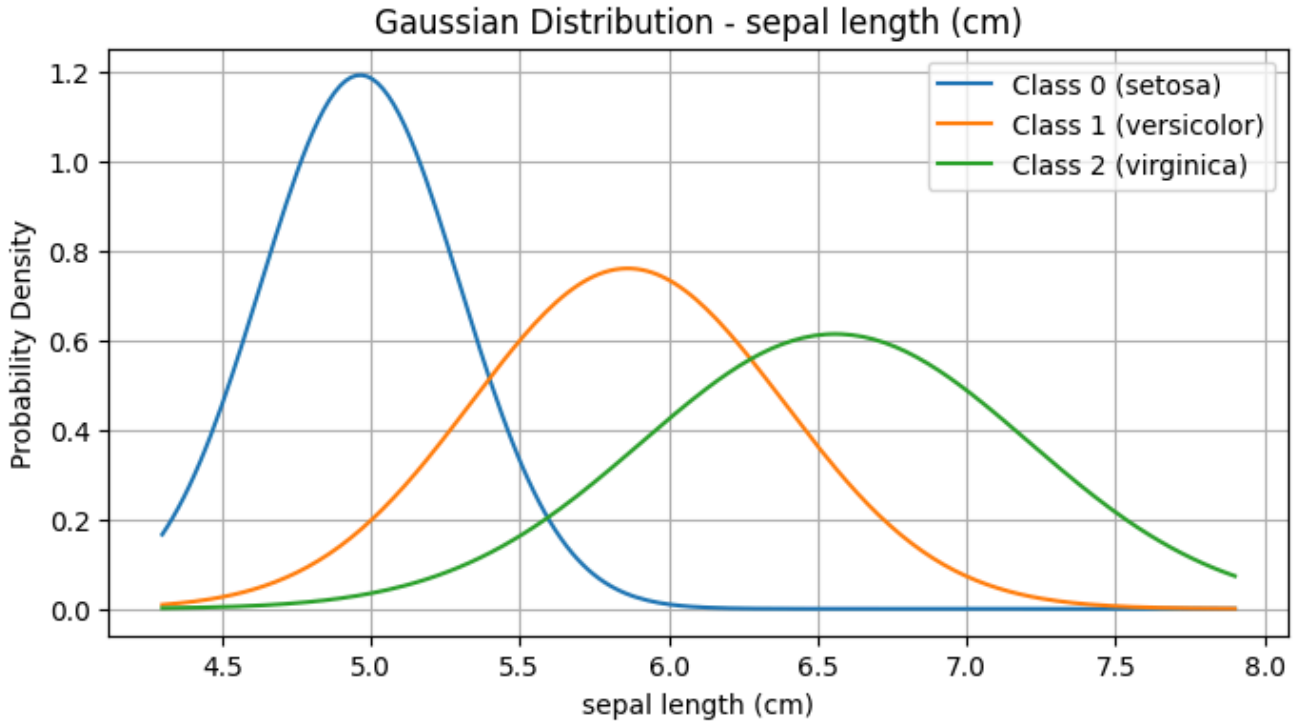
Predicted Class: $C = 0$ (Iris-setosa)

Code Implementation

```
import pandas as pd
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

iris = load_iris()

data = pd.DataFrame(iris.data, columns=iris.feature_names)
data['Species'] = iris.target

X = data.drop("Species", axis=1)
y = data['Species']
le = LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

gnb = GaussianNB()

gnb.fit(X_train, y_train)

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

feature_names = iris.feature_names
num_features = len(feature_names)
num_classes = len(np.unique(y))

X_np = X.to_numpy()

for feature_index in range(num_features):
    feature_name = feature_names[feature_index]
    x_vals = np.linspace(X_np[:, feature_index].min(), X_np[:, feature_index].max(),
200)

    plt.figure(figsize=(8, 4))

    for cls in range(num_classes):
        mean = gnb.theta_[cls, feature_index]
        std = np.sqrt(gnb.var_[cls, feature_index])
        y_vals = norm.pdf(x_vals, mean, std)
        plt.plot(x_vals, y_vals, label=f"Class {cls} ({iris.target_names[cls]})")

    plt.title(f"Gaussian Distribution - {feature_name}")
    plt.xlabel(feature_name)
    plt.ylabel("Probability Density")
    plt.legend()
    plt.grid(True)
```

```
plt.show()
```




```
y_pred = gnb.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"The Accuracy of Prediction on Iris Flower is: {accuracy}")
```

From Geek4Geek